

Software Design Diagram Generator

1. Stack at a glance

Layer	Choice
Frontend language and framework	TypeScript, React, Tailwind CSS
Frontend support libraries	Vite, React Router, shadcn/ui, TanStack Query, supabase-js, openapi-typescript, SVG pan/zoom viewer
Diagram rendering	Kroki (text description in, SVG/PNG/PDF out)
Backend language and framework	Python 3.12, FastAPI, Pydantic
AI orchestration	LangChain and LangGraph
LLM	Google Gemini Flash (free tier); exact model and fallback model(make use Groq)
Database	PostgreSQL on Supabase; SQLAlchemy 2 and Alembic
File storage	Supabase Storage
Authentication	Supabase Auth (email and password, Google) with authorisation in FastAPI
Background jobs and scheduled jobs	Jobs table in Postgres, in-process worker in the API
Email	Transactional provider through its HTTPS API (Brevo suggested); custom SMTP for Supabase auth mails
Export (initial)	PNG, SVG, PDF. Later: draw.io XML and PlantUML text (Sprint 3)
Repository, CI	GitHub monorepo, GitHub Actions
Hosting	Vercel (frontend), Render (backend), Supabase (DB, storage, auth), UptimeRobot
Team communication	Slack (mandatory course tool)

2. Frontend

TypeScript, React and Tailwind are decided. The supporting libraries below are defaults.

Library	What it does here
Vite	Build tool and dev server. The app sits entirely behind login, so server-side rendering is not needed.
React Router	Pages: login, project list, project workspace, admin dashboard.
shadcn/ui	Ready-made Tailwind components (dialogs, tables, buttons) copied into our code, with no heavy dependency.
TanStack Query	Fetching and caching API data; also polls generation status (“Queued”, “Generating diagram 2 of 3”) and notifications.
supabase-js	Login client for Supabase Auth (email and password, Google). Its token is sent to FastAPI on every request.
openapi-typescript	Generates TypeScript types from the FastAPI schema.
Library	What it does here
SVG pan/zoom viewer	Lets users navigate large diagrams returned as SVG.
React Flow, ELK.js	Not used with the recommended Kroki approach. They come back only if the group chooses Option B in Section 9.

3. Backend (Python, LangChain, LangGraph)

3.1 Components

Component	Role
FastAPI	Web framework. Asynchronous, validates requests automatically, generates the OpenAPI schema.
Pydantic	Defines every data shape once. The same schemas tell Gemini what JSON to return, which gives structured extraction.
SQLAlchemy 2 and Alembic	Database access and versioned migrations (TE3).
LangChain	Gemini connection (<code>langchain-google-gemai</code>), prompts and structured output.
LangGraph	Runs the pipeline as a graph of steps that share one state, with retries. See 5.2.
Job worker	Runs generation jobs and daily scheduled jobs. See 5.3.
Email sender	Sends verification, invitation, deletion-warning and notification emails through a provider's HTTPS API (TE5). See 5.4.
structlog	Structured logs with metadata only.

3.2 LangGraph pipeline

The pipeline in the concept poster maps naturally to a graph. Keep it small in Sprint 1 and pin library versions, because LangChain and LangGraph change quickly.

Sprint 1: `split_requirements -> extract_items (actors, entities, processes, flows/stores, interactions, relationships) -> merge -> generate_use_case -> mark_assumptions`
Later: `+ validate_requirements (flags, S2) -> recommend_types -> generate_class/activity/sequence/dfd/component -> validate_diagram (FR29)`

Everything that talks to Gemini goes through the single AI-service module (Section 10). Extraction must fit the 60 s median target (NFR1), so measure it early.

4. Hosting: Render, Vercel, UptimeRobot

4.1 Backend on Render (free web service)

- The FastAPI app runs in Docker (or as a plain Python service) and deploys automatically on merge to `main`. A failed deployment is visible to the team and the previous one can be restored (TE2).
- `/health` also runs a trivial database query, which keeps both the Render service awake (with the monitor) and the Supabase project from pausing.
- **UptimeRobot** calls `/health` every 10 minutes and doubles as the availability measurement (NFR7a).
- Free web services spin down after 15 minutes without traffic and take about a minute to wake up; the frontend must show the “server is starting” message (FR30c).
- Free instances have 512 MB RAM and 0.1 CPU. Watch memory when LangChain and PDF/DOCX parsing (Sprint 2) are loaded together.
- **750 free instance hours per month are shared across the whole Render workspace.** One always-awake API uses about 720, so we cannot run a second always-on service (for example a self-hosted Kroki) on Render.
- No persistent disk and no separate background workers on the free tier (see 5.3).

4.2 Frontend on Vercel: a problem to solve now

Vercel's Hobby (free) plan does not support collaboration on private repositories. A Hobby team cannot deploy from a private repository in a GitHub organisation, and on Hobby only commits authored by the team owner are deployed. With ten members each committing under their own account, most merges would not deploy.

Options: (a) make the repository public (Vercel says collaboration is free for public repos) — check the course allows it and that no secrets are committed; (b) pay for Vercel Pro; (c) host the frontend on another free host, which changes SRS constraint C2; (d) a GitHub Actions workaround that deploys with one person's token — it works but sidesteps the restriction, so treat it as a last resort. Suggested: (a).

5. Database and file storage (Supabase)

- **PostgreSQL** on Supabase holds accounts' project data, requirement statements, extracted items, diagrams, versions (latest 20 per diagram), flags, logs, jobs and feedback. It is encrypted at rest (NFR3a).
- **SQLAlchemy 2 with Alembic** gives typed models and versioned migrations (TE3). Use Supabase's connection pooler; choose session or transaction mode at setup (VERIFY prepared-statement behaviour with the async driver).
- **Supabase Storage** holds uploaded PDF and DOCX files (Sprint 2) and cached diagram SVG snapshots, so no separate storage service is needed.
- **Row Level Security** is enabled on every table with no browser-facing policies, so the public key can read nothing. Only the backend accesses data.
- **Storage is measured** so FR25c can report it against the 1 GB cap (TE3).
- **Free-plan limits that matter:** 500 MB database, 1 GB file storage, 50 MB maximum upload (our 10 MB limit fits), 2 active projects, no automatic backups, and projects are paused after 7 days of low activity. The health-route query prevents pausing; the nightly dump covers backups.

6. Authentication (Supabase Auth)

You asked whether Supabase Auth can do our auth on the free tier. **Yes, with adjustments.** It covers sign-up with verification, Google sign-in, login, logout and reset, and the free plan allows 50,000 monthly active users. Admin actions (add, deactivate, delete users) go through Supabase's admin API called from our backend with a

secret key. Some SRS rules do not map cleanly:

Our requirement	Free-tier reality	What we do
Verification and reset emails (FR28a, FR28e)	The built-in email service only delivers to addresses that belong to the project's team, at about 2 messages per hour. Nobody else can sign up until custom SMTP is configured. With custom SMTP the default limit is 30 new users per hour.	Configure custom SMTP (Brevo suggested). Mandatory before anyone outside the team can sign up.
Password of at least 8 characters with a letter and a digit	As far as we know, minimum length and required character types are configurable in Auth settings. VERIFY	Set it there and validate in the frontend.
Session expires after 24 h of inactivity (NFR4)	Supabase has an inactivity-timeout setting, but projects on the free plan report it as Pro-only. VERIFY in the dashboard	FastAPI stores "last active" per user and rejects tokens idle for over 24 h.
5 failed logins lock the account for 15 minutes (FR28c)	As far as we know, only per-IP rate limits exist and there is no per-account lockout. VERIFY	Route the login call through our backend so it can count failures, or relax the requirement (SRS change).
Verification link valid 24 h, reset link single-use and valid 1 h	As far as we know, one expiry setting covers email links. VERIFY	If it cannot be split, adjust the SRS.
Deactivate (FR23), delete (NFR5c), admin adds user (FR23b), inactivity purge (NFR5b)	Available through the admin API using the secret key.	Implement in the backend (secret key never sent to the browser).

How auth fits the architecture. The browser signs in through Supabase and sends the resulting token to FastAPI on every request. FastAPI verifies the token and enforces roles and "not found" responses from our own tables. The browser never queries our data tables directly. This is the only way to guarantee FR24e (role changes apply on the next request) and NFR4b. Because the token travels in a request header, no cross-site cookie setup or Vercel rewrite is needed.

7. Diagram rendering and editing (Kroki)

Status: ON HOLD — the group decides. Kroki as the rendering tool is decided. How users edit diagrams is not. The recommended option (A) is described so the group can accept or change it.

7.1 What Kroki is

Kroki is an HTTP service that turns diagram text (PlantUML, Graphviz, Mermaid, D2 and 30+ others) into images. It returns SVG, PNG or PDF depending on the diagram type. It is available as a free public service (kroki.io) or self-managed. It returns a **picture**, not an editable canvas, which is why editing needs a design.

7.2 Principle: the model is the diagram

In every option, the source of truth is the JSON model (FR16: diagram elements reference model items). Editing changes the model; the picture is regenerated from it.

Click element in SVG -> side panel shows properties + source requirements (FR17a)
edits: rename / add / delete / change relationship / properties (validated against notation rules)
Draft model in the browser
"Preview" (debounced or on Apply): backend model -> PlantUML / Graphviz text -> Kroki -> SVG + inject element IDs, warnings, assumptions style "Save": new versions stored (model JSON + SVG snapshot); revision number checked (FR24f)

7.3 The options

	A. Kroki + interactive overlay (recommended)	B. Own renderer in the browser	C. draw.io editor embedded
Display	Kroki SVG	React Flow (or maxGraph) + ELK.js layout, custom UML/DFD symbols	Embedded diagrams.net iframe fed with our .drawio XML
Click element → traceability (FR17a)	Yes, through element IDs in the SVG	Native	Partly; embed events need checking
Add, delete, rename, relationships, properties (FR12a-c, e, f)	Yes, through side panel, toolbar and inline rename	Yes, on canvas	Yes, but users can draw anything; invalid UML is caught only afterwards
Drag to move (FR12d)	No. Layout is automatic.	Yes	Yes
Notation correctness (DR1)	High: Kroki does the drawing	We must draw all six notations ourselves	Shape libraries exist; we must supply x/y coordinates
Assumption style and warnings	Styled in the source plus overlay badges	Native	Cell styles
Kroki still needed?	Yes	No (only for PlantUML text export)	No
Effort	Lowest	Highest (Sequence and Component need custom layout)	Medium, several unknowns
Fits Sprint 1 (14 days)?	Yes	Tight	Risky

Option D (edit the PlantUML text directly) is useful only as an advanced panel later: it breaks the link between diagram elements and model IDs, so it cannot be the main editor.

Why A is recommended: with four two-week sprints for 155 stories, A is the only option that gets notation right for free and leaves time for everything else. Its one real loss is drag-to-move. If the team insists on real drag-and-drop, choose B and drop Kroki as the display layer; that needs a strong frontend pair for the six symbol sets.

7.4 SRS changes if Option A is chosen

- **FR12d (move/reposition):** reword to "reorder and group elements and change layout direction; positions are automatic". The FR15b edge case about manual positions being lost becomes irrelevant.
- **FR12 draft semantics stay:** edits are held in a local draft until Save; "Preview" shows the re-render.

- **FR13a (regenerate part of a diagram)** works on model elements: the user selects elements, LangGraph regenerates those items, and we re-render.
- Log these as a new item in the Decision Log.

7.5 Solutions for the other Kroki risks

Risk	Solution
Element IDs in the SVG (click, warning badges, assumptions)	Give each PlantUML element an alias equal to its model ID. As far as we know, recent PlantUML versions tag entities with that alias in the SVG (VERIFY on the PlantUML version Kroki ships). For Graphviz, the <code>id</code> attribute goes straight into the SVG. The backend post-processes the SVG (adds <code>data-model-id</code> , warning and assumption styling).
Assumption look must survive every export (DR2b)	Post-processing writes styles inline (dashed stroke etc.), not as page CSS, so PNG, SVG and PDF all carry them.
DFD notation (DR1e)	Use Graphviz through Kroki for DFDs only: its HTML-like labels can draw the open-ended data store, plus circles and squares. PlantUML stays for the other five types. The renderer picks the engine per diagram type; both go through the same Kroki endpoint. Decide in Sprint 3.
Hosting, privacy, reliability	Keep the Kroki URL in an environment variable. Start on public kroki.io and add one line to the Privacy Policy (diagram text leaves our systems). Cache renders: hash the diagram text, store the SVG in Supabase Storage, re-render only when the text changes. This helps NFR1 and NFR6 and covers Kroki being down (FR30). Later, self-host on a free host outside Render (candidates not yet researched).
PNG (2×, 8,000 px cap) and PDF (A4)	Produce them ourselves from the stored SVG (for example with CairoSVG in Python) so we control scale, pixel cap, A4 fit and landscape switch. Every format comes from the same SVG, so exports match what the user sees. VERIFY CairoSVG handles PlantUML's SVG.
Combined project report (FR21)	Embeds the stored SVG snapshot of each diagram's latest version.

Build the renderer behind one interface, `render(model, diagram_type) → {svg, element_map}`. If the spike fails for one diagram type, we can swap just that type for a custom renderer without touching the rest.

8. AI provider (Groq) and the AI-service module

- **Provider:** Groq on the free tier (DECIDED). The exact model and a fallback model are OPEN.
- **TE4 module:** one module wraps Groq through LangChain; provider, model and API keys are configuration; keys are never sent to the browser; logs contain metadata only, never document content.
- **Hooks:** rate-limit queuing and retry and failure reporting.
- **Multi-key failover:** only enabled if the provider-terms check (O2, reported done) allows it. Record the outcome in the Decision Log before building.
- **Consent:** no content is sent to Groq until the user's consent is recorded with timestamp and policy version. The Privacy Policy states that content goes to an external provider.
- **Structured output:** Pydantic schemas define the extraction shape, and Groq returns validated JSON.

9. Export

Format	Sprint	How
PNG (2× scale, 8,000 px cap on longest side)	1 (FR18a)	Rasterised from the stored SVG
SVG	2 (FR18b)	The stored SVG with inline styles
PDF on A4, landscape if wider than tall, scaled to one page	2 (FR18c)	Converted from the stored SVG
Editable XML (.drawio)	3 (FR18d)	Serialised from the model; warnings recorded as metadata
PlantUML text (.puml)	3 (FR18e)	The same PlantUML text the renderer already builds; DFDs use rectangle, ellipse and database shapes with a header comment naming the mapping
One-click export in the default format	4 (FR18f)	PNG unless the user changed the default

For Sprint 1 and 2 only PNG, SVG and PDF are in scope. The assumption style must appear in every format once DR2b lands in Sprint 3.